SHAHID BEHESHTI UNIVERSITY
Faculty of Mathematical Sciences
Computer Science

Learning to See

CNN Design, Inspection and Architecture Experiments on CIFAR-10

Neural Networks | Fall 2026

Student	Sajjad Ranjbar
Student ID	405522012
Experiment platform	PyTorch, CPU; four execution threads
Date	11 November 2026

Abstract

I compare four compact convolutional classifiers under a common ten-epoch training budget on CIFAR-10. The study changes depth, activation function, and convolution type while preserving the data split and training procedure. Model and checkpoint selection use validation macro F1. Run 02, selected at epoch 10, achieves 76.70% test accuracy and 0.7656 test macro F1 in one final test inference pass. The models are inspected through intermediate activations and a separate validation-only occlusion experiment. These are short, single-seed experiments: they establish a reproducible comparison within this budget, not a globally optimal model.

1 Data, preprocessing and reproducibility

CIFAR-10 contains 60,000 RGB images of size 32×32 from ten balanced classes. The official 50,000 training examples are shuffled with a PyTorch generator seeded at 42. The first 5,000 indices form validation; the other 45,000 form training. The official 10,000 test examples remain reserved until architecture and checkpoint selection are finished. Saved split indices and their SHA-256 digest make the partition auditable.

Leakage prevention. The split is established before channel statistics are calculated. Only the 45,000 training images contribute to the per-channel population mean and standard deviation, using float64 accumulation after scaling bytes to $[0, 1]$:

$$\mu_c = \frac{1}{M} \sum_i x_{ic}, \qquad \sigma_c = \sqrt{\frac{1}{M} \sum_i x_{ic}^2 - \mu_c^2}, \qquad z_{ic} = \frac{x_{ic} - \mu_c}{\sigma_c}.$$

Here M counts all training pixels in a channel. The measured statistics are:

Training RGB mean	$(0.491350, 0.482161, 0.446597)$
Training RGB standard deviation	$(0.247049, 0.243554, 0.261684)$

Validation and test images use these same fixed training statistics. Computing them from all 60,000 examples would leak held-out input information into preprocessing.

Training uses a random 32×32 crop after padding by four pixels, random horizontal flips, tensor conversion and normalization. Evaluation uses only tensor conversion and normalization. An image tensor has shape $(3, 32, 32)$; PyTorch expects channels before height and width, whereas ordinary RGB image arrays use height, width, channels. A batch is $(N, 3, 32, 32)$.

The official download host was denied by the cloud network policy. The data were therefore obtained from a pinned GitHub mirror. All six original batch files and the metadata file matched the MD5 values distributed with `torchvision.datasets.CIFAR10` before loading; additional SHA-256 file digests and the mirror commit are stored in the provenance log. The mirror was used for data only, not for model code or performance results.

2 Baseline and training procedure

The baseline consists of three standard convolution blocks with output channels 16, 32 and 64. Each block applies a 3×3, stride-one, padding-one convolution, batch normalization, ReLU and 2×2 max pooling. Bias is omitted from convolutions followed by batch normalization. The spatial progression is

$$(3, 32, 32) \to (16, 16, 16) \to (32, 8, 8) \to (64, 4, 4) \to (64, 1, 1) \to 64 \to 10.$$

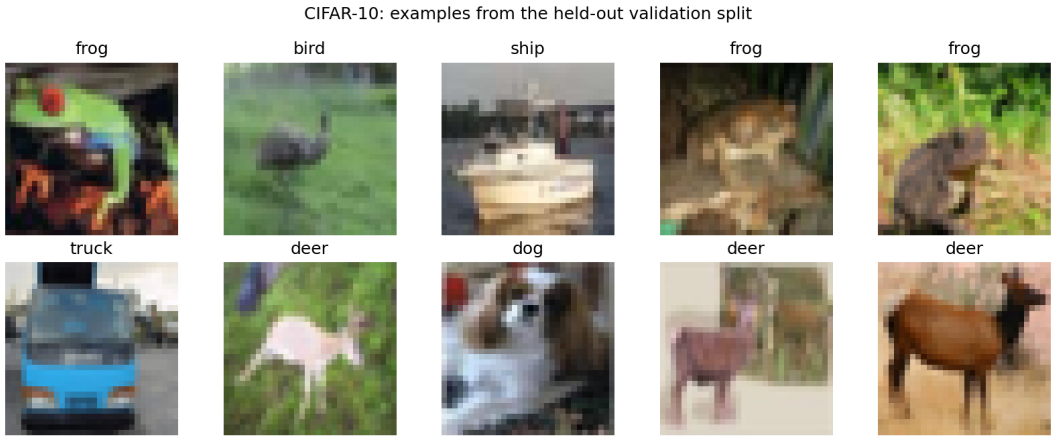


Figure 1: Ten validation examples. Labels are ground truth; images are displayed after undoing normalization.

Adaptive average pooling supplies the $(64, 1, 1)$ representation. The classifier is a 64-unit hidden linear layer, ReLU, dropout 0.3, and a final 10-unit linear layer. The baseline has 28,506 trainable parameters.

The output is ten raw class logits. Cross-entropy internally combines the log-softmax and negative log-likelihood operations, so a final model softmax is unnecessary. For visual confidence values only, softmax converts logits into probabilities. It is not a calibration guarantee.

Convolutions learn shared local pattern detectors, nonlinear activations make the overall mapping nonlinear, and pooling reduces spatial cost and increases the area of the input summarized by later features. Removing pooling retains fine detail but increases activation memory and computation; adaptive pooling still provides a fixed classifier interface.

Table 1: Training controls shared by all four configurations.

Setting	Value
Training / validation / test	45,000 / 5,000 / 10,000
Epochs per configuration	10, full training split each epoch
Batch size	128
Optimizer	AdamW
Initial learning rate / weight decay	0.003 / 0.0001
Schedule	Cosine annealing, $T_{\max} = 10$, minimum 0
Random seed	42; reset for every run
Classifier hidden units / dropout	64 / 0.3
Checkpoint selection	Highest validation macro F1; lower loss breaks ties

Each epoch records mean example-weighted cross-entropy and accuracy for training and validation, plus validation macro F1 and wall time. Training mode enables dropout and updates batch-normalization statistics; evaluation mode disables dropout and uses stored statistics. Gradients are disabled for all evaluation passes. Checkpoints are chosen using validation only. Training curves use augmented images with dropout, so their accuracy is not a perfectly matched comparison to clean validation accuracy.

3 Configurable architecture and controlled comparisons

The general `ConfigurableCNN` receives a dictionary of convolution blocks rather than fixing their number in code. Each block specifies output channels, kernel size, stride, padding, convolution type, optional batch normalization, activation, and optional pooling. Both ReLU and GELU are supported. Adaptive average pooling to 1×1 means that changing depth or input spatial size does not require manually changing the classifier input shape. Hidden dimension and dropout are configurable as well.

The alternate convolution type is depthwise-separable: a grouped spatial convolution filters each input channel separately, then a 1×1 pointwise convolution mixes channels. Its convolution weights scale as $C_{\text{in}}k^2 + C_{\text{in}}C_{\text{out}}$, compared with $C_{\text{in}}C_{\text{out}}k^2$ for a standard convolution. This reduces parameters but also changes the model's representational constraints; a parameter saving does not guarantee an accuracy improvement or a wall-time improvement.

Before training the alternatives, shape checks exercised standard and separable convolutions, ReLU and GELU, variable depth and hidden size, stride two, a 5×5 kernel, disabled batch normalization/pooling, and 32×32 and 40×40 inputs. Loading identical weights into the explicit baseline and its configurable equivalent produced identical logits.

Table 2: Predeclared interventions. All blocks use kernel/stride/padding 3/1/1, batch normalization and max pooling by two. The separable pointwise convolutions use 1/1/0.

Run	Block output channels	Convolution	Activation
01	16 / 32 / 64	Standard	ReLU
02	16 / 32 / 64 / 96	Standard	ReLU
03	16 / 32 / 64	Standard	GELU
04	16 / 32 / 64	Depthwise + pointwise	ReLU

The depth hypothesis is that another block improves the learned representation; it also adds a 96-channel stage and another downsampling step, so this is not a pure width experiment. The activation hypothesis is that GELU's smoother nonlinearity improves optimization. The efficiency hypothesis is that separable convolutions retain useful performance with fewer weights. Every run resets the random seed, shuffled data-loader generator, optimizer and schedule. Different model shapes still consume random numbers differently, so this is a shared protocol, not a promise of identical initialization across architectures.

Table 3: Measured comparison. Accuracy and loss are from each run's macro-F1-selected checkpoint, not independent best values. Wall time includes training and validation.

Run	Parameters	Epoch	Val. acc. (%)	Val. loss	Val. macro F1	Time (s)
01	28,506	9	67.68	0.9009	0.6716	116.1
02	86,042	10	77.58	0.6363	0.7732	120.9
03	28,506	9	70.16	0.8373	0.6981	127.6
04	8,101	10	58.02	1.1802	0.5741	116.4

Run 02 has the highest validation macro F1 (0.7732) under the fixed protocol.

Run 02 changes macro F1 by +0.1017 versus the baseline and uses 86,042 parameters.

Run 03 changes macro F1 by +0.0265 versus the baseline and uses 28,506 parameters.

Run 04 changes macro F1 by −0.0975 versus the baseline and uses 8,101 parameters.

In particular, run 04 did not improve baseline validation macro F1. This is a measured unsuccessful change under this budget, not proof that the architecture can never work.

The separable model reduces trainable parameters by 71.6%. Its accuracy and measured time must

be considered together with that saving. No multi-seed statistical significance is established by these comparisons.

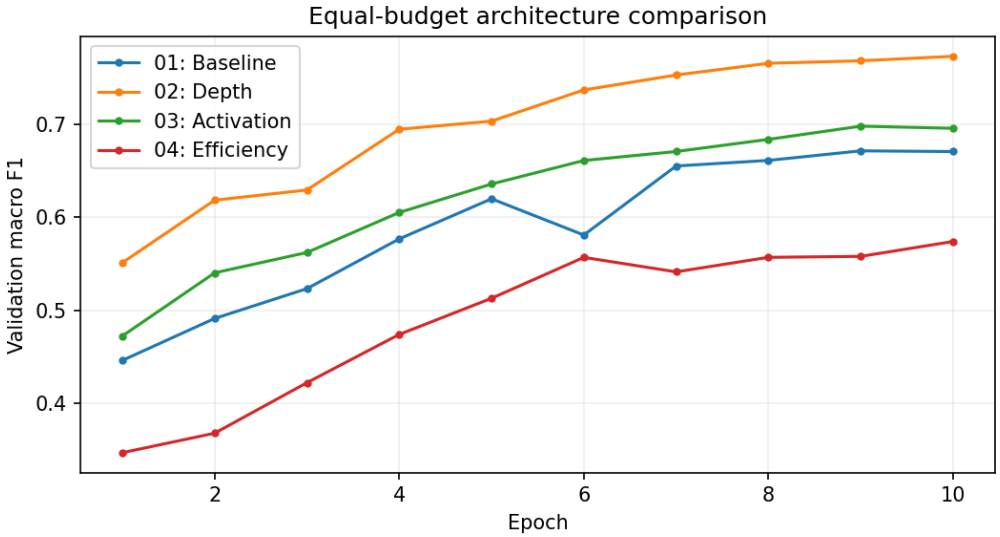


Figure 2: Validation macro F1 across the same ten-epoch budget. The official test split is not involved in these curves.

4 Learning curves and generalization

Run 01. At epoch 10, training/validation loss is 0.9617/0.8957 and accuracy is 65.70%/67.72%. The best observed validation loss is 0.8957. There is no clear late validation-loss increase exceeding 0.02; this short run does not establish full convergence.

Run 02. At epoch 10, training/validation loss is 0.7040/0.6363 and accuracy is 75.63%/77.58%. The best observed validation loss is 0.6363. There is no clear late validation-loss increase exceeding 0.02; this short run does not establish full convergence.

Run 03. At epoch 10, training/validation loss is 0.8921/0.8275 and accuracy is 68.35%/70.10%. The best observed validation loss is 0.8275. There is no clear late validation-loss increase exceeding 0.02; this short run does not establish full convergence.

Run 04. At epoch 10, training/validation loss is 1.2648/1.1802 and accuracy is 54.28%/58.02%. The best observed validation loss is 1.1802. There is no clear late validation-loss increase exceeding 0.02; this short run does not establish full convergence.

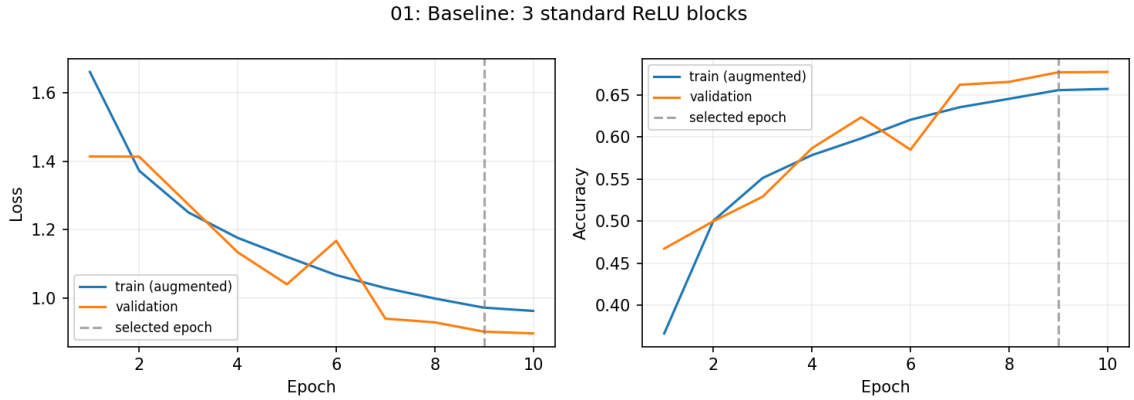


Figure 3: Run 01: baseline loss and accuracy. The vertical line marks the epoch selected by validation macro F1.

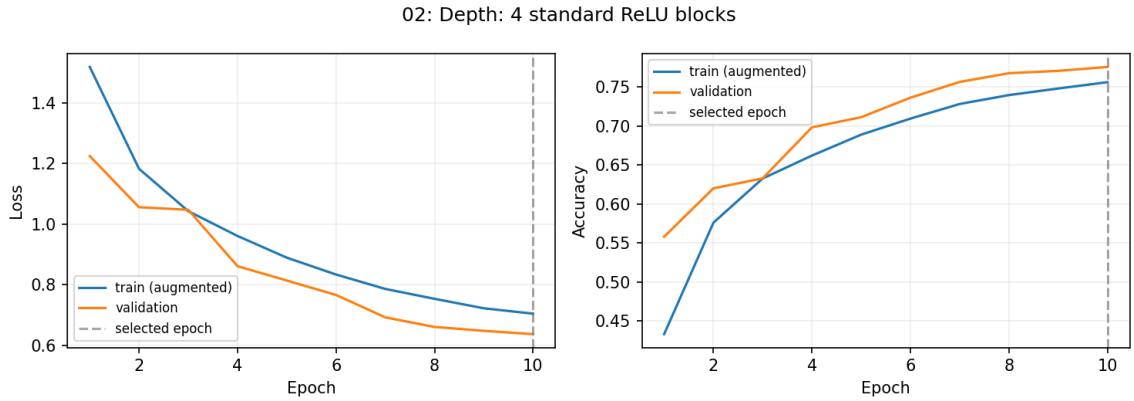


Figure 4: Run 02: additional depth and a 96-channel last block, with the same training budget.

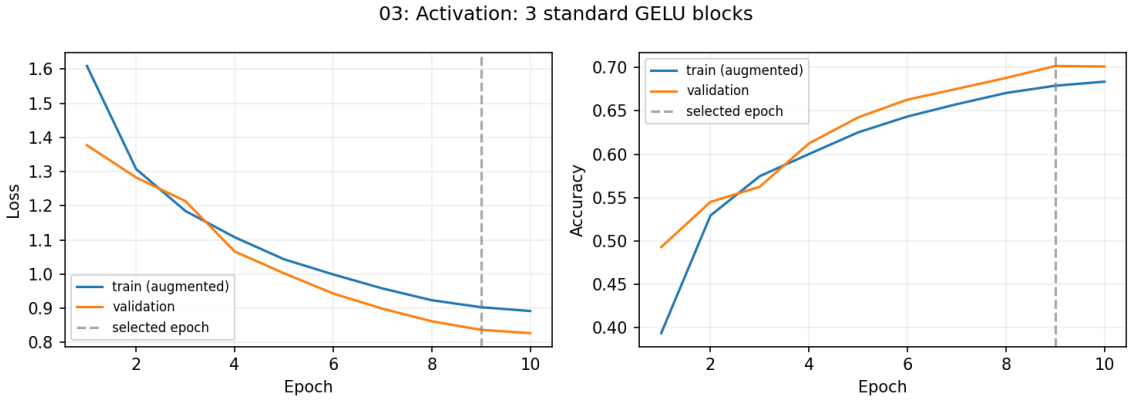


Figure 5: Run 03: GELU replaces ReLU in convolution blocks and the hidden classifier.

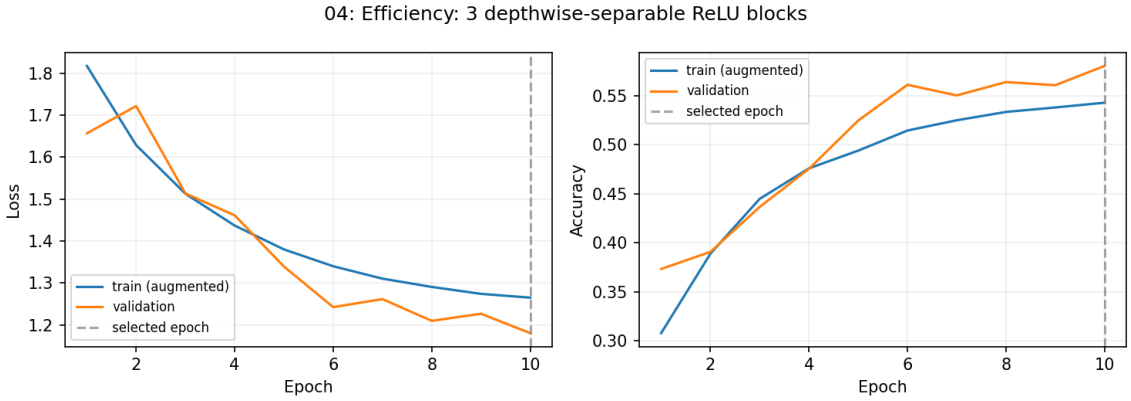


Figure 6: Run 04: depthwise-separable convolutions replace standard convolutions.

5 Selected model architecture

The winning run is 02 (Depth: 4 standard ReLU blocks), checkpoint epoch 10. No architecture change or retraining follows the test results. The model has 86,042 trainable parameters.

Table 4: Selected convolution blocks. Output shapes omit the batch dimension. Each block has batch normalization and max pooling by two; activation is relu.

Block	Channels	Type	Kernel/stride/padding	Output (C, H, W)
1	$3 \to 16$	Standard	3/1/1	(16,16,16)
2	$16 \to 32$	Standard	3/1/1	(32,8,8)
3	$32 \to 64$	Standard	3/1/1	(64,4,4)
4	$64 \to 96$	Standard	3/1/1	(96,2,2)

The post-feature representation is adaptively averaged to one spatial cell, flattened, projected to 64 hidden units with the selected activation, passed through dropout 0.3, and projected to ten logits. This table describes the actual saved checkpoint, not a separately proposed architecture.

6 Intermediate activations and feature maps

Forward hooks capture the outputs of entire convolution blocks, after activation and pooling. The examples are the first airplane and first cat in the validation split, selected by true label before visual inspection. For each, eight channels from the first and last blocks are shown. The baseline and selected model are both inspected, allowing a direct view of how the studied representations differ.

The selected model's block-output shapes are $(16, 16, 16) \to (32, 8, 8) \to (64, 4, 4) \to (96, 2, 2)$. Across the two selected-model inputs and all blocks, 10 near-constant channel responses are recorded (a response count, not a count of distinct filters). In the baseline airplane image, first-block channels 0, 2, 3 and 4 emphasize a narrow vertical central structure, consistent with a local contrast or edge response. Channels 1, 5 and 7 distribute responses more broadly, including image borders; channel 6 contains more scattered local responses. For the cat image the strong regions move toward upper-left and lower image structures, which shows that these are input-dependent filters rather than fixed class masks. Baseline late channels 0, 2 and 6 emphasize middle/lower coarse cells for the airplane, while channels 1, 4 and 7 show distinct upper/lower spatial responses. These are visual descriptions, not evidence that any particular channel detects an airplane, a cat or a named body part. Early feature maps retain finer spatial variation in edges, colour regions and texture; later maps are coarser summaries with larger receptive fields. A channel can be approximately constant because its preferred pattern is absent, because ReLU suppresses it, or because its response is spatially uniform. Spatial standard deviation below 10^{-5} is logged as a near-constant response for this input. A single image cannot establish that a filter is permanently inactive. Downsampling discards exact positions and fine structures; not every semantic distinction survives it.

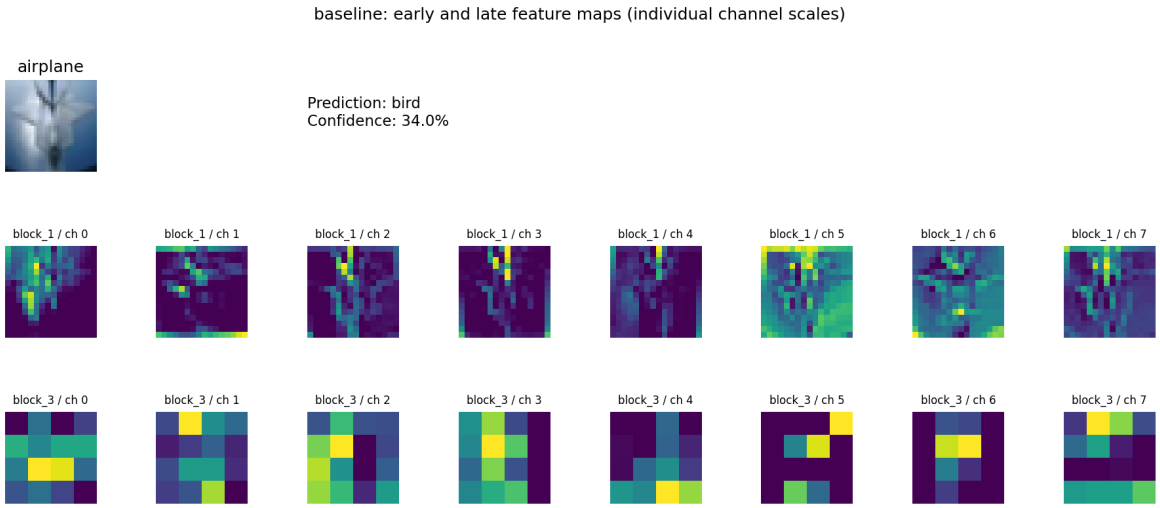


Figure 7: Baseline activations for a validation airplane. Top: input and prediction. Middle: first block. Bottom: last block. Each channel has its own intensity scale.

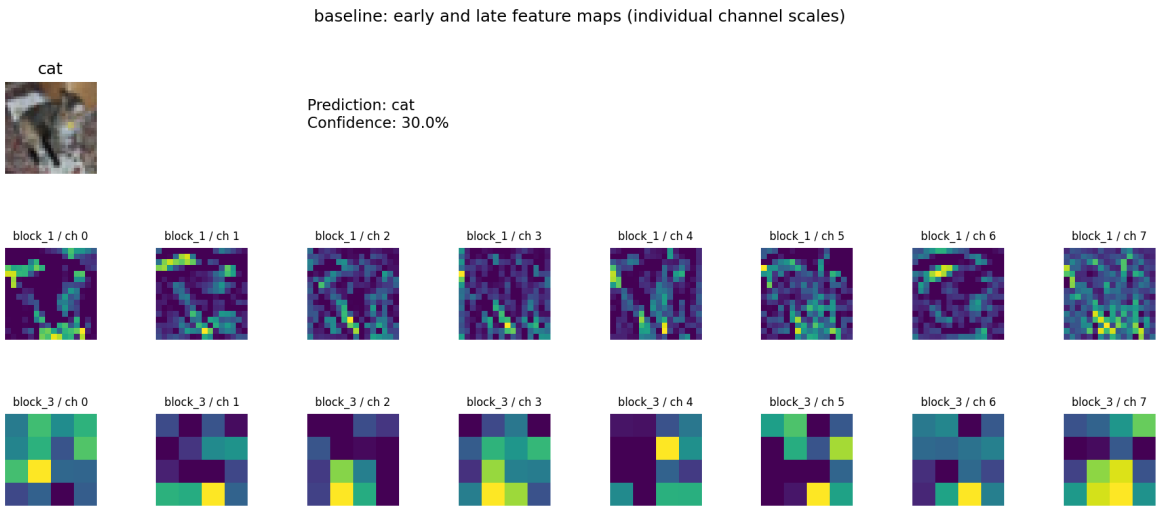


Figure 8: Baseline activations for a validation cat. The same filter indices act on different input content.

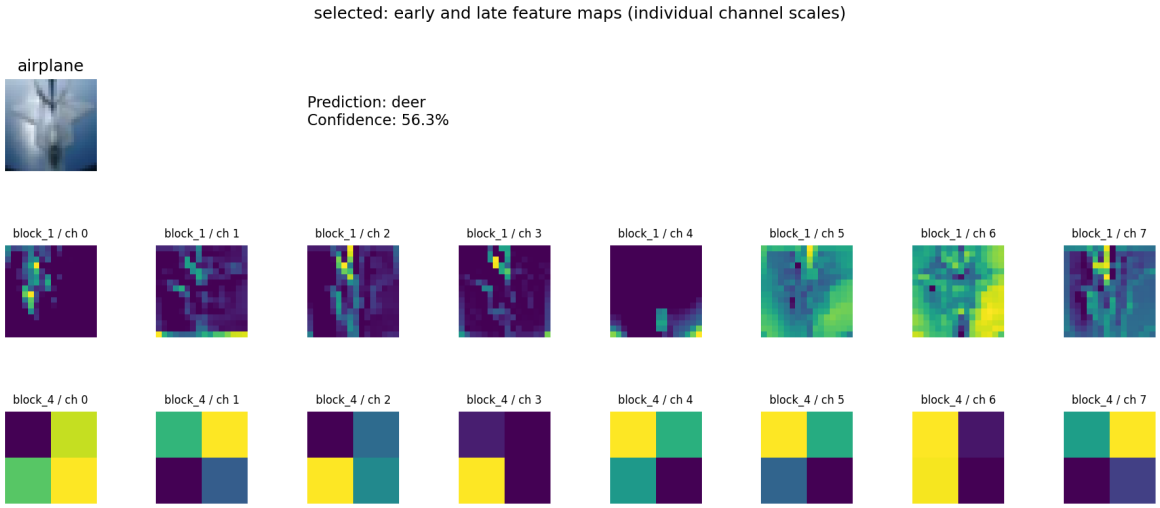


Figure 9: Selected-model activations for the same validation airplane. Brightness across panels is not an absolute activation comparison.

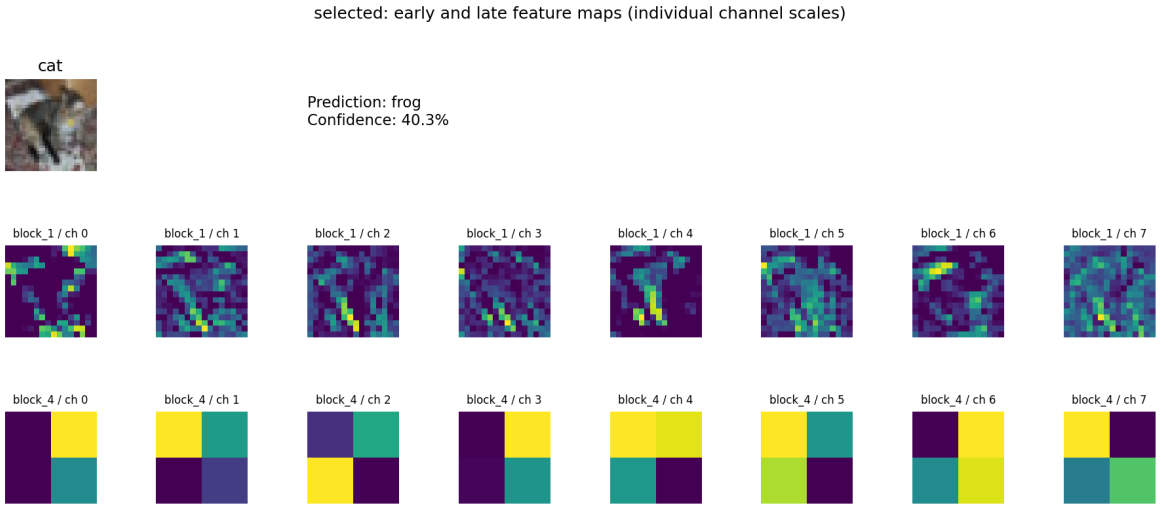


Figure 10: Selected-model activations for the same validation cat. Visual differences do not by themselves identify a filter's semantic meaning.

7 Multiclass metrics and final test evaluation

For class c, let TP_c, FP_c and FN_c denote true positives, false positives and false negatives. The reported metrics are

$$\text{accuracy} = \frac{\sum_c TP_c}{N}, \quad P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = \frac{2P_cR_c}{P_c + R_c}.$$

A zero denominator is assigned score zero by the metric implementation. Macro metrics average the ten class scores equally. Weighted metrics average them by true-class support. Each official test class has 1,000 examples, so macro and weighted precision, recall and F1 coincide here. Weighted recall also equals accuracy in single-label multiclass classification.

For cat, a dog predicted as cat is a false positive and a cat predicted as dog is a false negative. Predicting cat very rarely but accurately can give high cat precision and low cat recall. Accuracy alone hides this pattern; macro F1 assigns each class equal weight, and the confusion matrix identifies the directions and magnitudes of class confusions.

The architecture and checkpoint epoch were frozen using validation macro F1 (0.7732). The test loader was then evaluated once for 10,000 examples. Every metric, the per-class report and the confusion matrix below reuse that same inference pass. No test result was used to select another model or training procedure. Raw labels and predictions are saved so these metrics can be independently recalculated without running the model again.

Table 5: Final aggregate test metrics, all on the same 10,000 predictions.

Metric	Value
Accuracy	0.7670
Macro precision	0.7651
Macro recall	0.7670
Macro F1	0.7656
Weighted precision	0.7651
Weighted recall	0.7670
Weighted F1	0.7656

Table 6: Per-class final test precision, recall and F1.

Class	Precision	Recall	F1	Support
airplane	0.7714	0.8100	0.7902	1000
automobile	0.8980	0.8800	0.8889	1000
bird	0.6905	0.6290	0.6583	1000
cat	0.5873	0.5350	0.5599	1000
deer	0.7423	0.7460	0.7441	1000
dog	0.6404	0.6750	0.6573	1000
frog	0.8011	0.8380	0.8192	1000
horse	0.8212	0.8130	0.8171	1000
ship	0.8622	0.8760	0.8690	1000
truck	0.8370	0.8680	0.8522	1000

By test F1, the easiest class is automobile (0.8889) and the hardest is cat (0.5599). The largest directed confusion is cat predicted as dog (193 examples). The confusion matrix supplies the specific errors hidden by the overall accuracy. Class-wise results are descriptive outcomes of the frozen model, not a further model-selection input.

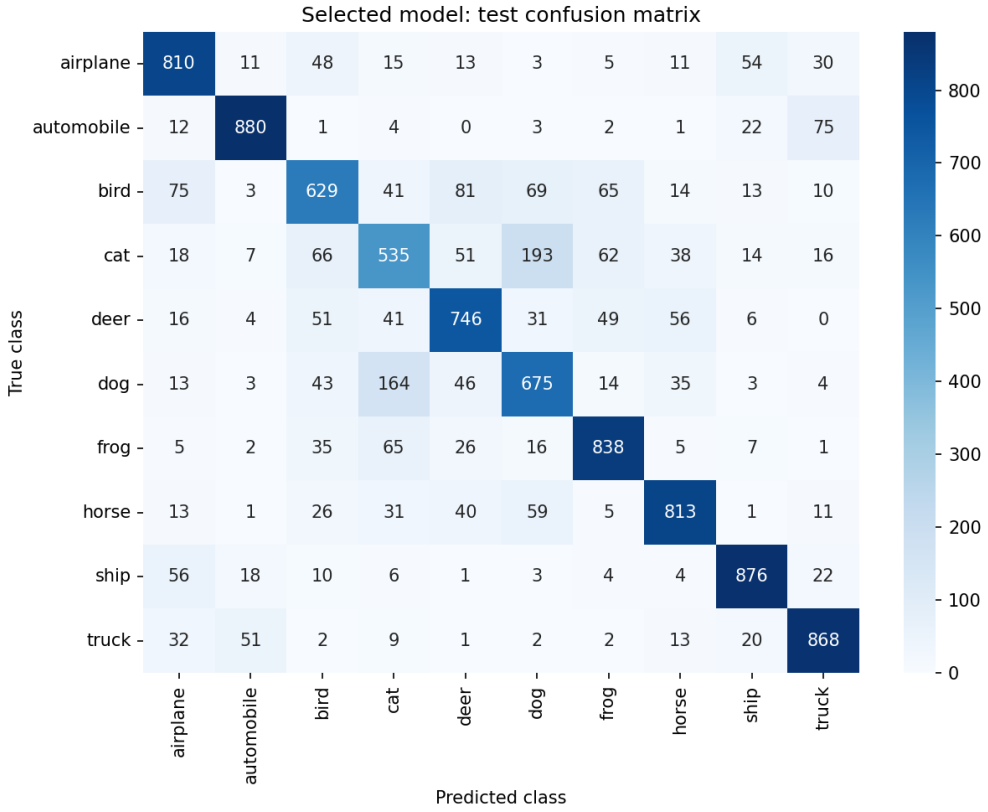


Figure 11: Final test confusion matrix. Rows are true labels and columns are predictions; all 10,000 examples are included.

8 Optional robustness experiment

The optional extension asks what happens when central visual evidence is removed. A fixed 12×12 centre patch covers 144 of 1,024 pixels (14.0625%). Values are set to zero in normalized space, which corresponds to the training RGB mean, not black. The selected weights are frozen and all 5,000 validation examples are evaluated. This uses validation only and does not add another test inference pass or alter model selection. Clean validation accuracy changes from 77.58% to 59.44% under occlusion, a decrease of 18.14 percentage points. Macro F1 changes from 0.7732 to 0.5888 (difference −0.1844). This probe measures sensitivity to one particular spatial corruption. It cannot establish



Figure 12: Clean validation images above and their occluded counterparts below. Titles show true class followed by predicted class.

robustness to all occlusions, shifts, noise or adversarial changes, and the centre location is not a causal explanation of what a CNN has learned.

9 Reflection and limitations

Among these interventions, run 02 has the largest absolute validation macro-F1 change versus baseline (+0.1017). The deeper model (run 02) scores 0.7732, compared with 0.6716 for baseline. This single comparison does not imply that increasing depth or width always improves validation performance; depth and the extra output width were changed together, and width alone was not isolated. The histories show any late validation deterioration directly; the final test evaluation does not diagnose which additional architecture should be tried. The experiment uses one seed and a short CPU budget. Reported differences can depend on initialization, the validation sample and training duration. No confidence interval or multi-seed significance claim is made. Comparing only four configurations cannot establish a global optimum. Lower training loss alone is not a sufficient selection criterion, and augmented training accuracy must be interpreted alongside clean validation metrics.

A useful next study would repeat the same validation-based comparison over several seeds and a larger common epoch budget, then investigate remaining class confusions. Those decisions must not repeatedly consult this already reported test set. Test results in this report are a final measurement of this completed selection procedure.

Reproduction and attribution

The executed notebook is `Noebooks/CIFAR10_CNN_Assignment.ipynb`. It contains the full model, training, metric and plotting code, all assignment answers, and outputs. Measured per-epoch CSV files, full configuration dictionaries, split indices, data provenance, environment versions, per-class metrics and final predictions are in `report/results/`. Figures are in `report/figures/`. Dependencies are pinned in `requirements.txt`; the README gives CPU installation and PDF build commands. Model checkpoints are local generated files excluded from Git; running the notebook reconstructs them.

The assignment prompt comes from the course repository. OpenAI Codex assisted with implementation, experiment execution, explanations and LaTeX preparation at the student's request. All performance values in this report were measured in the saved runs; none are borrowed from published models. The named student should review the work and attribution before submission.

References

[1] A. Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. 2009. https://www.cs.toronto.edu/~kriz/cifar.html.

[2] PyTorch contributors. PyTorch documentation. https://pytorch.org/docs/stable/.

[3] Torchvision contributors. CIFAR10 dataset implementation and checksums. https://docs.pytorch.org/vision/stable/datasets/generated/torchvision.datasets.CIFAR10.html.

[4] Scikit-learn contributors. Model evaluation and classification metrics. https://scikit-learn.org/stable/modules/model_evaluation.html.

[5] CIFAR-10 batch-file mirror, `0Colin/cifar-10-batches-py`, commit `48ece5ce490482885f7a9a492fa031c1de79519c`. https://github.com/0Colin/cifar-10-batches-py.